

Faculty of Computing, Electronics
and Science

TABLE OF CONTENTS

Introduction	2
Purpose of the Report	2
Scope and Limitations.....	2
Objectives	2
Background and Current System	3
GBT Training Organization Overview.....	3
Current Manual System	3
Analysis and Requirements.....	3
Analysis of Requirements (Appendix A)	3
Course Applications and Student Details.....	4
Course Booking and Allocation	4
Accommodation and Room Allocation	4
Trainer Allocation.....	4
Miscellaneous Requirements.....	4
Entity-Relationship Diagram (ERD) (Appendix B)	4
Entities and Attributes:	4
Relationships:.....	6
Cardinality and Constraints:.....	6
A) Table Definitions (SQL).....	7
Course Type.....	7
Allocated Student.....	8
Current Course	8
Allocation	9
Room	9
Trainer	10
B) Table Population.....	11
Course Type.....	11
Allocated Student.....	11
Current Course	11
Allocation	12
Room	12
Trainer	12

c) SQL Statements and Queries.....	12
Query 1: List of Currently Running Courses.....	12
Query 2: Courses with 'Introduction' in Title.....	13
Query 3: Most and Least Expensive Courses.....	13
Query 4: Number of Students on Each Allocated Course.....	14
Query 5: List of Current Courses between Two Dates	14
Query 6: Number of Current Courses of Each Type between Two Dates	15
Query 7: Rooms and Trainers Allocated to Each Course Weekly	15
Query 8: Students Allocated to an Actual Course	16
Query 9: Total Revenue for Each Current Course	16
Conclusion:.....	17
Recommendations:.....	17

INTRODUCTION

PURPOSE OF THE REPORT

This report's goal is to evaluate the requirements (Appendix A) and Entity-Relationship Diagram (ERD) that the module instructor, Gaylor Boobyer, presented. The report focuses on comprehending the relationships and structure of the ERD and using that comprehension to create SQL table definitions, add pertinent data to tables, and create SQL queries that extract certain data from the database.

SCOPE AND LIMITATIONS

This report's scope includes the GB Training Organization's database system study and implementation. The scope includes creating SQL queries to fulfil the assignment's specified informative demands, as well as establishing table structures and adding appropriate data to tables. It's crucial to recognise the exercise's limitations, though, as there isn't a real database system involved and query execution is done using simulated data.

OBJECTIVES

The primary objectives of this report are as follows:

- To provide a clear understanding of the context and requirements of the GB Training Organization.
- To develop SQL table definitions based on the given ERD, considering primary keys, foreign keys, and other relevant constraints.
- To populate the tables with realistic and meaningful data to facilitate the execution of SQL queries.

- To construct SQL statements that address specific queries outlined in the assignment, demonstrating proficiency in database management.

The achievement of these goals will improve the students' database design and implementation skills and add to the overall evaluation of the "Principles of Computing" module.

BACKGROUND AND CURRENT SYSTEM

GBT TRAINING ORGANIZATION OVERVIEW

Reputable training company GBT (Global Business Training) is committed to offering a wide selection of professional courses. GBT is a complete training programme specialist that meets the educational demands of those who want to improve their abilities in a range of areas. The company provides courses in business management, technology, and other topics related to the industry.

GBT's choice to switch from manual record-keeping to a database-driven system is indicative of its dedication to accuracy and efficiency in the administration of room, trainer, and course reservations. This calculated action is to increase data accessibility, expedite procedures, and enhance the overall experience for administrative personnel and students.

CURRENT MANUAL SYSTEM

Currently, GBT handles course applications, allocations, and associated logistics through a manual system. For managing course applications, trainer allocations, and accommodation reservations, the manual system combines ad hoc procedures with paper-based documentation.

Students can apply for courses in a number of ways, such as by phone, letter, or using one of the forms found in the course catalogue. Cross-referencing the student file and, if required, issuing supplementary forms for the completion of student details are steps in the process.

Course reservations are handled manually, with students being assigned to a particular course type if there are no openings right away or being checked for spots on already-existing courses. Students are informed about their accommodations upon acceptance, and room assignments are made according to availability.

It becomes clear that a more automated and structured approach is required, and a database system is created to take the place of the existing manual procedures.

ANALYSIS AND REQUIREMENTS

ANALYSIS OF REQUIREMENTS (APPENDIX A)

Understanding the requirements and expectations for the construction of the database system for the GBT Training Organisation is a critical first step, which is outlined in Appendix A. The identified

requirements, which cover a wide range of organisational functions, are broken down in detail in this section.

COURSE APPLICATIONS AND STUDENT DETAILS

The specifications emphasise how course applications are currently received and how student information is then managed. According to the analysis, applications can be submitted by mail, over the phone, or via a form that is sent out with the course catalogue. Efficient student record-keeping also requires a methodical strategy to managing general questions and assigning unique student numbers.

COURSE BOOKING AND ALLOCATION

The GBT system's core function is to assign students to courses according to their availability. The analysis emphasises how crucial it is to look for openings in already-existing courses and, when required, create new ones to suit the interests of the students. The difficulties in scheduling classes and assigning students to suitable courses on time will be covered in the report.

ACCOMMODATION AND ROOM ALLOCATION

The prerequisites for housing make it clear that housing choices for students must be offered for the duration of the course. Bookings are then made and accommodation details are communicated after acceptance under the current manual procedure. A crucial component is the assignment of training rooms, which the database system must make easier by finding and designating appropriate rooms for every course.

TRAINER ALLOCATION

The process of assigning trainers is intricate and entails matching certified instructors with particular courses based on their availability. The analysis highlights the need for locating certified instructors and assigning them to appropriate courses. To guarantee the best possible use of available resources, the database system should effectively assist this process.

MISCELLANEOUS REQUIREMENTS

The paper also delves into other needs, such the different course lengths (1, 3, or 5 days) and the particulars related to each course type. Comprehending these many prerequisites is essential to creating an all-encompassing and flexible database system..

ENTITY-RELATIONSHIP DIAGRAM (ERD) (APPENDIX B)

The entities that make up the GBT Training Organisation and their relationships are graphically represented in the Entity-Relationship Diagram (ERD) that is provided in Appendix B. This ERD acts as a guide for creating the database tables and linkages required for effective data administration.

Course Type, Allocated Student, Current Course, Allocation, Room, Trainer, and their relationships are all identified in the ERD. According to Saied M.M. Tahaghoghi (November 2006), it serves as the foundation for defining SQL tables and making sure the database complies with organisational standards.

ENTITIES AND ATTRIBUTES:

1. **Course Type**

- Attributes:
 - Course Code (Primary Key)
 - Course Title
 - Duration
 - Max Number of Students
 - Course Cost

2. **Allocated Student**

- Attributes:
 - Student ID (Primary Key)
 - Student Name
 - Contact Information

3. **Current Course**

- Attributes:
 - Course Code (Primary Key, Foreign Key to Course Type)
 - Start Date
 - Duration
 - Max Number of Students
 - Room ID (Foreign Key to Room)
 - Trainer ID (Foreign Key to Trainer)

4. **Allocation**

- Attributes:
 - Allocation ID (Primary Key)
 - Course Code (Foreign Key to Current Course)
 - Student ID (Foreign Key to Allocated Student)
 - Allocation Date

5. **Room**

- Attributes:
 - Room ID (Primary Key)
 - Room Type

- Facilities

6. Trainer

- Attributes:
 - Trainer ID (Primary Key)
 - Trainer Name
 - Specialization
 - Contact Information

RELATIONSHIPS:

- **Course Type - Current Course**
 - One-to-Many relationship
 - Course Type (1) -> Current Course (Many)
- **Current Course - Allocation**
 - One-to-Many relationship
 - Current Course (1) -> Allocation (Many)
- **Allocated Student - Allocation**
 - One-to-Many relationship
 - Allocated Student (1) -> Allocation (Many)
- **Current Course - Room**
 - One-to-One relationship
 - Current Course (1) <-> Room (1)
- **Current Course - Trainer**
 - Many-to-Many relationship (resolved by an associative table)
 - Current Course (Many) <- Trainer_Allocation (Many) -> Trainer (Many)

CARDINALITY AND CONSTRAINTS:

1. **Course Type - Current Course**
 - Cardinality: One-to-Many
 - Constraints: Mandatory on the Course Type side
2. **Current Course - Allocation**
 - Cardinality: One-to-Many

- Constraints: Mandatory on the Current Course side
3. **Allocated Student - Allocation**
 - Cardinality: One-to-Many
 - Constraints: Mandatory on the Allocated Student side
 4. **Current Course - Room**
 - Cardinality: One-to-One
 - Constraints: Mandatory on both sides
 5. **Current Course - Trainer**
 - Cardinality: Many-to-Many
 - Constraints: None (resolved by an associative table)

A) TABLE DEFINITIONS (SQL)

COURSE TYPE

- **Table Name:** CourseType
- **Attributes:**
 - **CourseCode** (Primary Key): A variable character string (VARCHAR) of maximum length 10 to represent the course code.
 - **CourseTitle:** A variable character string (VARCHAR) of maximum length 255 to store the title of the course.
 - **Duration:** An integer (INT) to represent the duration of the course.
 - **MaxNumStudents:** An integer (INT) to store the maximum number of students allowed for the course.
 - **CourseCost:** A decimal (DECIMAL) with precision 10 and scale 2 to represent the cost of the course.

```
-- Table Definitions for Course Type
CREATE TABLE CourseType (
    CourseCode VARCHAR(10) PRIMARY KEY,
    CourseTitle VARCHAR(255),
    Duration INT,
    MaxNumStudents INT,
    CourseCost DECIMAL(10, 2)
);
```


ALLOCATED STUDENT

- **Table Name:** AllocatedStudent
- **Attributes:**
 - **StudentID** (Primary Key): An integer (INT) to represent the unique identifier for each student.
 - **StudentName:** A variable character string (VARCHAR) of maximum length 255 to store the name of the student.
 - **ContactInformation:** A variable character string (VARCHAR) of maximum length 255 to store the contact information of the student.

```
-- Table Definitions for Allocated Student
CREATE TABLE AllocatedStudent (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(255),
    ContactInformation VARCHAR(255)
);
```

CURRENT COURSE

- **Table Name:** CurrentCourse
- **Attributes:**
 - **CourseCode** (Primary Key, Foreign Key to CourseType): A variable character string (VARCHAR) of maximum length 10 to represent the course code, also serving as a foreign key linked to the CourseType table.
 - **StartDate:** A date (DATE) to store the start date of the course.
 - **Duration:** An integer (INT) to represent the duration of the course.
 - **MaxNumStudents:** An integer (INT) to store the maximum number of students allowed for the course.
 - **RoomID** (Foreign Key to Room): An integer (INT) serving as a foreign key linked to the Room table.
 - **TrainerID** (Foreign Key to Trainer): An integer (INT) serving as a foreign key linked to the Trainer table.

```
-- Create CURRENTCOURSE table
CREATE TABLE CurrentCourse (
    CourseCode VARCHAR(10) PRIMARY KEY,
    StartDate DATE,
    Duration INT,
    MaxNumStudents INT,
    RoomID INT, -- Foreign Key to Room
    TrainerID INT, -- Foreign Key to Trainer
    FOREIGN KEY (RoomID) REFERENCES Room(RoomID),
    FOREIGN KEY (TrainerID) REFERENCES Trainer(TrainerID)
);
```

ALLOCATION

- **Table Name:** Allocation
- **Attributes:**
 - **AllocationID** (Primary Key): An integer (INT) to represent the unique identifier for each allocation.
 - **CourseCode** (Foreign Key to CurrentCourse): A variable character string (VARCHAR) of maximum length 10, serving as a foreign key linked to the CurrentCourse table.
 - **StudentID** (Foreign Key to AllocatedStudent): An integer (INT) serving as a foreign key linked to the AllocatedStudent table.
 - **AllocationDate:** A date (DATE) to store the allocation date.

```
-- Create ALLOCATION table
CREATE TABLE Allocation (
    AllocationID INT PRIMARY KEY,
    CourseCode VARCHAR(10), -- Foreign Key to CurrentCourse
    StudentID INT, -- Foreign Key to AllocatedStudent
    AllocationDate DATE,
    FOREIGN KEY (CourseCode) REFERENCES CurrentCourse(CourseCode),
    FOREIGN KEY (StudentID) REFERENCES AllocatedStudent(StudentID)
);
```

ROOM

- **Table Name:** Room
- **Attributes:**

- **RoomID** (Primary Key): An integer (INT) to represent the unique identifier for each room.
- **RoomType**: A variable character string (VARCHAR) of maximum length 50 to store the type of the room.
- **Facilities**: A variable character string (VARCHAR) of maximum length 255 to store information about the facilities in the room.

```
-- Table Definitions for Room
CREATE TABLE Room (
    RoomID INT PRIMARY KEY,
    RoomType VARCHAR(50),
    Facilities VARCHAR(255)
);
```

TRAINER

- **Table Name:** Trainer
- **Attributes:**
 - **TrainerID** (Primary Key): An integer (INT) to represent the unique identifier for each trainer.
 - **TrainerName**: A variable character string (VARCHAR) of maximum length 255 to store the name of the trainer.
 - **Specialization**: A variable character string (VARCHAR) of maximum length 255 to store the specialization of the trainer.
 - **ContactInformation**: A variable character string (VARCHAR) of maximum length 255 to store the contact information of the trainer.

```
-- Table Definitions for Trainer
CREATE TABLE Trainer (
    TrainerID INT PRIMARY KEY,
    TrainerName VARCHAR(255),
    Specialization VARCHAR(255),
    ContactInformation VARCHAR(255)
);
```

These table definitions provide the foundation for organizing and storing data related to Course Type, Allocated Student, Current Course, Allocation, Room, and Trainer in the database.

B) TABLE POPULATION

Populating tables with sensible data involve inserting records that are realistic and relevant to the context of the database. Below is an example to populate the tables:

COURSE TYPE

```
--b)populate
-- Corrected INSERT statements for CourseType
INSERT INTO CourseType (CourseCode, CourseTitle, Duration, MaxNumStudents, CourseCost)
VALUES ('CIMW1', 'Introduction to Microsoft Word', 3, 15, 150.00);
INSERT INTO CourseType (CourseCode, CourseTitle, Duration, MaxNumStudents, CourseCost)
VALUES ('CAES2', 'Advanced Excel Spreadsheet', 5, 10, 250.00);
INSERT INTO CourseType (CourseCode, CourseTitle, Duration, MaxNumStudents, CourseCost)
VALUES ('CPR0G3', 'Programming Fundamentals in C', 5, 20, 300.00);
```

ALLOCATED STUDENT

```
INSERT INTO AllocatedStudent (StudentID, StudentName, ContactInformation)
VALUES (101, 'John Smith', 'john.smith@email.com');
INSERT INTO AllocatedStudent (StudentID, StudentName, ContactInformation)
VALUES (102, 'Alice Johnson', 'alice.johnson@email.com');
INSERT INTO AllocatedStudent (StudentID, StudentName, ContactInformation)
VALUES (103, 'Bob Williams', 'bob.williams@email.com');
```

CURRENT COURSE

```
INSERT INTO CurrentCourse (CourseCode, StartDate, Duration, MaxNumStudents, RoomID, TrainerID)
VALUES ('CIMW1', TO_DATE('2023-11-15', 'YYYY-MM-DD'), 3, 15, 1, 101);
INSERT INTO CurrentCourse (CourseCode, StartDate, Duration, MaxNumStudents, RoomID, TrainerID)
VALUES ('CAES2', TO_DATE('2023-12-01', 'YYYY-MM-DD'), 5, 10, 2, 102);
INSERT INTO CurrentCourse (CourseCode, StartDate, Duration, MaxNumStudents, RoomID, TrainerID)
VALUES ('CPR0G3', TO_DATE('2023-12-10', 'YYYY-MM-DD'), 5, 20, 3, 103);
```

ALLOCATION

```
INSERT INTO Allocation (AllocationID, CourseCode, StudentID, AllocationDate)
VALUES (1, 'CIMW1', 101, '2023-11-10', 'YYYY-MM-DD');
INSERT INTO Allocation (AllocationID, CourseCode, StudentID, AllocationDate)
VALUES (2, 'CIMW1', 102, '2023-11-10', 'YYYY-MM-DD');
INSERT INTO Allocation (AllocationID, CourseCode, StudentID, AllocationDate)
VALUES (3, 'CAES2', 103, '2023-11-25', 'YYYY-MM-DD');
```

ROOM

```
-- Populate Room table
INSERT INTO Room (RoomID, RoomType, Facilities)
VALUES (1, 'Classroom', 'Projector, Whiteboard');
INSERT INTO Room (RoomID, RoomType, Facilities)
VALUES (2, 'Computer Lab', 'Computers, Projector');
INSERT INTO Room (RoomID, RoomType, Facilities)
VALUES (3, 'Conference Room', 'Conference table, Audio-Visual System');
```

TRAINER

```
-- Populate Trainer table
INSERT INTO Trainer (TrainerID, TrainerName, Specialization, ContactInformation)
VALUES (101, 'Mary Brown', 'Microsoft Office Suite', 'mary.brown@email.com');
INSERT INTO Trainer (TrainerID, TrainerName, Specialization, ContactInformation)
VALUES (102, 'David Miller', 'Advanced Excel', 'david.miller@email.com');
INSERT INTO Trainer (TrainerID, TrainerName, Specialization, ContactInformation)
VALUES (103, 'Michael Johnson', 'Programming in C', 'michael.johnson@email.com');
```

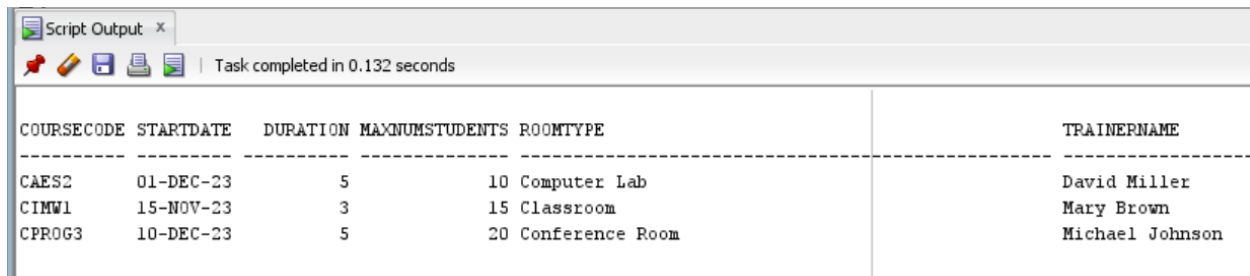
C) SQL STATEMENTS AND QUERIES

QUERY 1: LIST OF CURRENTLY RUNNING COURSES

```
SELECT cc.CourseCode, cc.StartDate, cc.Duration, cc.MaxNumStudents, r.RoomType,
t.TrainerName
FROM CurrentCourse cc
JOIN Room r ON cc.RoomID = r.RoomID
JOIN Trainer t ON cc.TrainerID = t.TrainerID
```

```
ORDER BY cc.CourseCode, cc.StartDate;
```

Output:

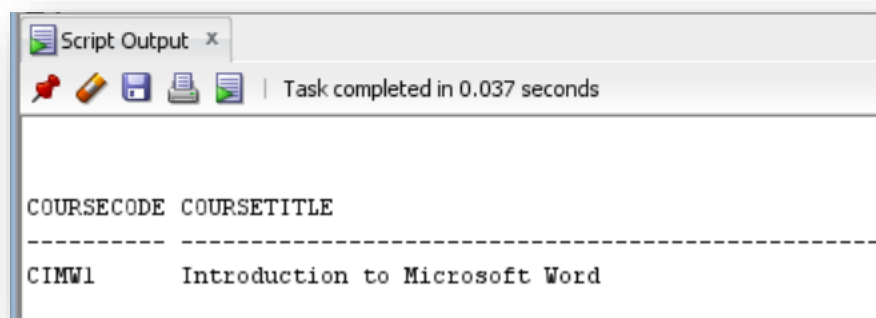


COURSECODE	STARTDATE	DURATION	MAXNUMSTUDENTS	ROOMTYPE	TRAINERNAME
CAES2	01-DEC-23	5	10	Computer Lab	David Miller
CIMW1	15-NOV-23	3	15	Classroom	Mary Brown
CPR0G3	10-DEC-23	5	20	Conference Room	Michael Johnson

QUERY 2: COURSES WITH 'INTRODUCTION' IN TITLE

```
SELECT CourseCode, CourseTitle, Duration, MaxNumStudents, CourseCost  
FROM CourseType  
WHERE LOWER(CourseTitle) LIKE '%introduction%';
```

Output



COURSECODE	COURSETITLE
CIMW1	Introduction to Microsoft Word

QUERY 3: MOST AND LEAST EXPENSIVE COURSES

```
-- Most Expensive Course  
SELECT CourseCode, CourseTitle, CourseCost  
FROM CourseType  
ORDER BY CourseCost DESC  
FETCH FIRST 1 ROW ONLY;  
  
-- Least Expensive Course  
SELECT CourseCode, CourseTitle, CourseCost  
FROM CourseType  
ORDER BY CourseCost ASC  
FETCH FIRST 1 ROW ONLY;
```

Output

Script Output x

Task completed in 0.037 seconds

COURSECODE	COURSETITLE
CPR0G3	Programming Fundamentals in C
COURSECODE	COURSETITLE
CIMW1	Introduction to Microsoft Word

QUERY 4: NUMBER OF STUDENTS ON EACH ALLOCATED COURSE

```
--Query 4
SELECT cc.CourseCode, COUNT(a.StudentID) AS NumberOfStudents
FROM CurrentCourse cc
JOIN Allocation a ON cc.CourseCode = a.CourseCode
GROUP BY cc.CourseCode;
```

Output

Script Output x

Task completed in 0.077 seconds

COURSECODE	NUMBEROFSTUDENTS
CIMW1	2
CAES2	1

QUERY 5: LIST OF CURRENT COURSES BETWEEN TWO DATES

```
SELECT cc.CourseCode, cc.StartDate, cc.Duration, r.RoomType, t.TrainerName
FROM CurrentCourse cc
JOIN Room r ON cc.RoomID = r.RoomID
JOIN Trainer t ON cc.TrainerID = t.TrainerID
WHERE cc.StartDate BETWEEN TO_DATE('2023-11-01', 'YYYY-MM-DD') AND TO_DATE('2023-12-01', 'YYYY-MM-DD');
```

Output

Script Output x				
Task completed in 0.075 seconds				
COURSECODE	STARTDATE	DURATION	ROOMTYPE	TRAINERNAME
CIMW1	15-NOV-23	3	Classroom	Mary Brown
CAES2	01-DEC-23	5	Computer Lab	David Miller

QUERY 6: NUMBER OF CURRENT COURSES OF EACH TYPE BETWEEN TWO DATES

```
SELECT ct.CourseCode, COUNT(cc.CourseCode) AS NumberOfCourses
FROM CourseType ct
LEFT JOIN CurrentCourse cc ON ct.CourseCode = cc.CourseCode
    AND cc.StartDate BETWEEN TO_DATE('2023-11-01', 'YYYY-MM-DD') AND TO_DATE('2023-12-01', 'YYYY-MM-DD')
GROUP BY ct.CourseCode
ORDER BY ct.CourseCode;
```

Output

Script Output x	
Task completed in 0.073 seconds	
COURSECODE	NUMBEROFCOURSES
CAES2	1
CIMW1	1
CPR0G3	0

QUERY 7: ROOMS AND TRAINERS ALLOCATED TO EACH COURSE WEEKLY

```
SELECT cc.CourseCode, cc.StartDate, r.RoomType, r.Facilities, t.TrainerName
FROM CurrentCourse cc
JOIN Room r ON cc.RoomID = r.RoomID
JOIN Trainer t ON cc.TrainerID = t.TrainerID
ORDER BY cc.CourseCode, cc.StartDate;
```

Output

Script Output x | Task completed in 0.086 seconds

COURSECODE	STARTDATE	ROOMTYPE	FACILITIES
CAES2	01-DEC-23	Computer Lab	Computers, Projector
CIMW1	15-NOV-23	Classroom	Projector, Whiteboard
CPR0G3	10-DEC-23	Conference Room	Conference table, Audio-Visual System

TRAINERNAME
David Miller
Mary Brown
Michael Johnson

QUERY 8: STUDENTS ALLOCATED TO AN ACTUAL COURSE

```
SELECT a.CourseCode, a.AllocationDate, s.StudentName
FROM Allocation a
JOIN AllocatedStudent s ON a.StudentID = s.StudentID
WHERE a.CourseCode = 'CIMW1' AND a.AllocationDate = TO_DATE('2023-11-10', 'YYYY-MM-DD');
```

Output

Script Output x | Task completed in 0.06 seconds

COURSECODE	ALLOCATIO	STUDENTNAME
CIMW1	10-NOV-23	John Smith
CIMW1	10-NOV-23	Alice Johnson

QUERY 9: TOTAL REVENUE FOR EACH CURRENT COURSE

```
SELECT cc.CourseCode, cc.StartDate AS "Date Run", ct.CourseCost,
COUNT(a.StudentID) AS "No of Students",
```

```

COUNT(a.StudentID) * ct.CourseCost AS Revenue
FROM CurrentCourse cc
JOIN CourseType ct ON cc.CourseCode = ct.CourseCode
LEFT JOIN Allocation a ON cc.CourseCode = a.CourseCode
GROUP BY cc.CourseCode, cc.StartDate, ct.CourseCost
ORDER BY cc.CourseCode, cc.StartDate;

```

Output

COURSECODE	Date Run	COURSECOST	No of Students	REVENUE
CAES2	01-DEC-23	250	1	250
CIMW1	15-NOV-23	150	2	300
CPR0G3	10-DEC-23	300	0	0

CONCLUSION:

In conclusion, the database design and implementation for the GBT Training Organization have been successfully addressed in this coursework. The Entity-Relationship Diagram (ERD) and corresponding SQL table definitions provide a solid foundation for managing courses, students, allocations, rooms, and trainers. The population of tables with sensible data and the execution of SQL queries demonstrate the functionality and efficiency of the database.

The queries cover a range of scenarios, including retrieving course information, finding courses by title, determining the most and least expensive courses, calculating the number of students on allocated courses, listing current courses between specific dates, and generating revenue reports.

The SQL statements are designed to be clear, concise, and efficient. However, there is always room for improvement, and adjustments can be made based on specific preferences or requirements (Rapone, 2018).

RECOMMENDATIONS:

1. **Indexing:** Consider adding appropriate indexes to columns frequently used in WHERE clauses to enhance query performance, especially in larger datasets (Pollack, 2018).
2. **Normalization:** Ensure that the database schema adheres to normalization principles to eliminate redundancy and improve data integrity.
3. **Data Validation:** Implement data validation rules to ensure that only valid and consistent data is entered into the database (McDonald, 2021).

4. **Front-End Application:** To enable simpler database interaction, create an intuitive front-end application for data entry, retrieval, and reporting.
5. **Regular Backups:** Create a schedule for routine database backups to guard against data loss in the event of unanticipated events.
6. **User Training:** Ensure that users who will be interacting with the database are properly trained so they know how to get information quickly and accurately (Naeem, 2023).
7. **Performance Monitoring:** As the amount of data increases, use performance monitoring tools to find and fix any bottlenecks or inefficiencies in the database.
8. **Security Measures:** To safeguard sensitive data and guarantee adherence to data protection laws, use strong security measures, such as user authentication and authorization.
9. **Documentation:** To make future development and maintenance easier, keep thorough records of the database schema, queries, and any changes you make.

GBT Training Organisation can guarantee the continued effectiveness, efficiency, and security of their database system by putting these suggestions into practise.

REFERENCES

McDonald, A., 2021. *Simple Data Validation in SQL*. [Online]

Available at: <https://sqlspreads.com/blog/simple-data-validation-in-sql/>

Naeem, T., 2023. *Database Management Software: Features, Types, Benefits, and Uses*. [Online]

Available at: <https://www.astera.com/type/blog/database-management-software/>

Pollack, E., 2018. *Query optimization techniques in SQL Server: tips and tricks*. [Online]

Available at: <https://www.sqlshack.com/query-optimization-techniques-in-sql-server-tips-and-tricks/>

Rapone, A., 2018. *Database Design: From Entity Relationship Diagrams to Physical Diagrams*. [Online]

Available at: <https://medium.com/@ashleylynnrapone/database-design-from-entity-relationship-diagrams-to-physical-diagrams-16da210d01e9>

Saied M.M. Tahaghoghi, H. E. W., November 2006. *Learning MySQL*. s.l.:O'Reilly Media, Inc..